Linux Kernel Development: Command line argument & /proc file system

__init module and __exit module

- After install module the program will run **init_module()** function as entry point as you may see in Hello World program. Alternatively, you can use the combination of **__init function_name** and declare it **module_init** later.

- When the module is removed the program will run **cleanup()** function before it remove itself. Like init_module you can use a combination of **__exit function_name** and declare it **module_exit** later.

Example

```
#include <linux/module.h>
#include <linux/printk.h>

int init_module(void)
{
    pr_info("Hello world 1.\n");
    return 0;
}

void cleanup_module(void)
{
    pr_info("Goodbye world 1.\n");
}

MODULE_LICENSE("GPL");
```

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/printk.h>

static int __init hello_2_init(void)
{
    pr_info("Hello, world 2\n");
    return 0;
}
static void __exit hello_2_exit(void)
{
    pr_info("Goodbye, world 2\n");
}

module_init(hello_2_init);
module_exit(hello_2_exit);

MODULE_LICENSE("GPL");
```

Command line arguments

```
... include list ...

int myInt = 3;
module_param(myint, int, 0);

char* myString = "hello";
module_param(myString, charp, 0);
MODULE_PARM_DESC(mystring, "A character string");

... function ...
```

- Kernel module can also receive command line arguments via the use of module_param() on the global variable and can be declare as shown above.
- module_param() takes 3 arguments which are the name of the variable, its type and permissions to /sys file. You can findout more about permissions here: https://github.com/torvalds/linux/blob/master/include/linux/moduleparam.h at line 115 to 138
- Additionally, you can describe the parameter with the use of MODULE_PARM_DESC(<variable name>, <description>)

Read and Write via /proc

- Alternatively, kernel module provide a method to read and write information from/to the process via the use of /proc
- /proc file system can respond to both read and write operation.
- You can use the command cat to read and print out the content of the file. in the case of /proc the command will trigger read operation.
- likewise the command like echo <message> | tee <file> which result in the <message> being written to the <file> will also trigger the write operation for /proc

Example command:

Read

```
$ cat /proc/<module_name>
```

Write

```
$ echo <message> | sudo tee <file>
```

Define /proc variable

```
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/uaccess.h>
#include <linux/version.h>

#if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 6, 0)
#define HAVE_PROC_OPS
#endif

#define PROCFS_MAX_SIZE 1024
#define PROCFS_NAME "buffer1k"

/* This structure hold information about the /proc file */
static struct proc_dir_entry *our_proc_file;
/* The buffer used to store character for this module */
static char procfs_buffer[PROCFS_MAX_SIZE];
/* The size of the buffer */
static unsigned long procfs_buffer_size = 0;
```

Define read operation

```
/* This function is called then the /proc file is read */
static ssize_t procfile_read(struct file *file_pointer, char __user *buffer,
                             size_t buffer_length, loff_t *offset)
{

    char s[13] = "HelloWorld!\n";
    int len = sizeof(s);
    ssize_t ret = len;
    if (*offset >= len || copy_to_user(buffer, s, len)) {
        pr_info("copy_to_user failed\n");
        ret = 0;
    } else {
        pr_info("procfile read %s\n", file_pointer->f_path.dentry->d_name.name);
        *offset += len;
    }
    return ret;
}
```

- The content on **buffer** will be the output

Define write operation

```
/* This function is called when the /proc file is written. */
static ssize_t procfile_write(struct file *file, const char __user *buff,
                              size_t len, loff_t *off)
{
    procfs_buffer_size = len;
    if (procfs_buffer_size >= PROCFS_MAX_SIZE)
        procfs_buffer_size = PROCFS_MAX_SIZE - 1;

    if (copy_from_user(procfs_buffer, buff, procfs_buffer_size))
        return -EFAULT;

    procfs_buffer[procfs_buffer_size] = '\0';
    *off += procfs_buffer_size;

    pr_info("procfile write %s\n", procfs_buffer);

    return procfs_buffer_size;
}
```

Register read and write operation

```
#ifdef HAVE_PROC_OPS // refer to define /proc variable page it’s basically check the version
static const struct proc_ops proc_file_fops = {
    .proc_read = procfile_read,
    .proc_write = procfile_write,
};
#else
static const struct file_operations proc_file_fops = {
    .read = procfile_read,
    .write = procfile_write,
};
#endif
```

Declare __init and __exit funciton

```
static int __init procfs2_init(void)
{
    our_proc_file = proc_create(PROCFS_NAME, 0644, NULL, &proc_file_fops);
    if (NULL == our_proc_file) {
        pr_alert("Error:Could not initialize /proc/%s\n", PROCFS_NAME);
        return -ENOMEM;
    }
    pr_info("/proc/%s created\n", PROCFS_NAME);
    return 0;
}
static void __exit procfs2_exit(void)
{
    proc_remove(our_proc_file);
    pr_info("/proc/%s removed\n", PROCFS_NAME);
}
module_init(procfs2_init);
module_exit(procfs2_exit);
MODULE_LICENSE("GPL");
```

Demo

Exercise

- Try create module that will print out (read operation) the content that is come from write operation from earlier. If no write operation has been done then print “helloworld”
- Note that both read and write function is call back function that would trigger when its corrosponding operation is being triggered. So the question is what will happened if both operation is being triggered at the same time and how to solve it?
- You can read more from: https://sysprog21.github.io/lkmpg/#read-and-write-a-proc-file